Generated from the Markdown study system. Labs with executable code remain in the `labs/` folder. This is study material, not a Cisco exam dump and not a pass guarantee.

CCNA Automation Local Lab Setup (Windows)

This document builds a **free or very-low-cost** lab you can reuse for every 200-901 CCNAAUTO v1.1 domain. You do not need physical Cisco gear. You will combine:

- a local Windows 11 workstation
- WSL2 Ubuntu for Linux, Bash, Ansible, and Git practice
- Python, VS Code, Postman, Docker Desktop, and Terraform on Windows or WSL
- **Cisco DevNet Sandbox** for real Cisco APIs, IOS XE, NETCONF, and RESTCONF

The exam is not a hardware-configuration test. It is a software, API, automation, and fundamentals exam. This lab matches that mix.

---

1. What to install

Tool	Where	Why you need it
Windows 11	Host	Your daily OS. Hyper-V/WSL2/Docker run here.
WSL2 + Ubuntu	Windows	Bash, Linux file layout, Ansible, realistic PATH/env labs.
Python 3.12+	Windows **and** Ubuntu	Scripts, `requests`, unit tests, parsing JSON/YAML/XML.
Git	Windows + Ubuntu	Version control objectives 1.7 and 1.8.
Visual Studio Code	Windows	Editor, Python debugger, integrated terminal, WSL remote.
Postman	Windows	Construct and inspect REST/RESTCONF requests without code.
Docker Desktop	Windows (WSL2 backend)	Images, Dockerfiles, local containers (4.6, 4.7).
Terraform	Windows or Ubuntu	Read/apply basic HCL (5.5, 5.6).
Ansible	Ubuntu (WSL)	Interpret and run simple playbooks (5.6, 5.8).
GitHub account	Cloud	Remote clone/push/pull and code-review practice.
Cisco DevNet account	Cloud	Sandbox, Code Exchange, Learning Labs, API docs.
Free Webex account	Cloud	Spaces, messages, memberships (3.9.b).

Optional later: Cisco Modeling Labs Personal (not required to start; useful for 5.3 and network diagrams). pyATS is free via pip when you reach infrastructure testing.

---

2. Installation steps (Windows 11)

Do these in order. After each tool, verify with the command shown.

2.1 Enable WSL2 and Ubuntu

In an **elevated** PowerShell window:

```
wsl --install -d Ubuntu
```

Reboot if Windows asks. After Ubuntu launches, create a UNIX username and password.

Verify:

```
wsl --status
wsl --set-default-version 2
```

Inside Ubuntu:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git python3 python3-venv python3-pip build-essential curl unzip
python3 --version
```

2.2 Install Python on Windows

Do **not** use the Microsoft Store stub that only opens the Store page. Download the official installer:

https://www.python.org/downloads/windows/

During setup:

1. Check **Add python.exe to PATH**.
2. Choose **Install Now** (or Customize and enable `pip` and `py launcher`).
3. Disable the Store app execution aliases if `python` still fails:
   Settings → Apps → Advanced app settings → App execution aliases → turn off `python.exe` and `python3.exe`.

Verify in a **new** PowerShell window:

```
python --version
python -m pip --version
```

2.3 Install Git for Windows

https://git-scm.com/download/win

Keep the default editor (or choose VS Code). Use **Git from the command line and also from 3rd-party software**. Prefer **OpenSSL** and **Checkout as-is, commit Unix-style line endings** for mixed Windows/WSL work.

```
git --version
```

2.4 Install Visual Studio Code

https://code.visualstudio.com/

Recommended extensions:

- Python (`ms-python.python`)
- Pylance

- Jupyter (optional)
- YAML
- Docker
- WSL (`ms-vscode-remote.remote-wsl`)
- REST Client or Thunder Client (optional if you prefer not to use Postman)

Open this folder:

`C:\Users\Reydel\Documents\04_Learning\CCNA Automation`

Then **File → Open Folder**. Use **Terminal → New Terminal** and pick PowerShell or Ubuntu (WSL).

2.5 Install Postman

https://www.postman.com/downloads/

Create a free Postman account. You will import requests for httpbin, RESTCONF, Meraki, Catalyst Center, and Webex.

Disable SSL verification for lab devices that use self-signed certificates: **Settings → General → SSL certificate verification → OFF** (lab use only).

2.6 Install Docker Desktop

https://www.docker.com/products/docker-desktop/

Enable the **WSL2 backend**. After install, confirm Ubuntu is listed under **Settings → Resources → WSL integration**.

```
docker version
docker run --rm hello-world
```

If Hyper-V/WSL virtualization is disabled, enable it in Windows Features and in firmware (VT-x/AMD-V).

2.7 Install Terraform

Windows (package manager):

```
winget install HashiCorp.Terraform
```

Or download from https://developer.hashicorp.com/terraform/install.

Ubuntu:

```
sudo apt update
sudo apt install -y gnupg software-properties-common
# Follow HashiCorp's current apt instructions:
# https://developer.hashicorp.com/terraform/install
terraform version
```

2.8 Install Ansible in Ubuntu (not natively on Windows)

```
sudo apt update
sudo apt install -y ansible
ansible --version
```

Ansible control nodes expect Linux. Running it in WSL is the correct Windows approach for this exam.

2.9 Create free cloud accounts

1. **GitHub:** https://github.com/signup
2. **Cisco ID / DevNet:** https://developer.cisco.com/
3. **DevNet Sandbox:** https://devnetsandbox.cisco.com/
4. **Webex developer portal:** https://developer.webex.com/

DevNet Sandbox availability changes. Always-On IOS XE (Catalyst 8000/9000), NSO, and Nexus labs are the usual starting points. Meraki, Catalyst Center, ISE, and SD-WAN sandboxes are sometimes taken offline for maintenance. If a tile is missing, use a reservable lab or your own API key on a non-production org.

---

3. Recommended folder structure

```
CCNA Automation/
├── 200-901-CCNAAUTO_v.1.1.pdf          # official blueprint (do not edit)
├── Automation-v2.0-Learning-Matrix.xlsx # original matrix (do not edit)
├── CCNAAUTO_Study_Tracker.xlsx
├── CCNAAUTO_COMPLETE_STUDY_GUIDE.md
├── CCNAAUTO_FINAL_REVIEW.md
├── CCNAAUTO_STUDY_PLAN.md
├── CCNAAUTO_PRACTICE_QUESTIONS.md
├── CCNAAUTO_OBJECTIVE_CHECKLIST.md
├── CCNAAUTO_LAB_SETUP.md
└── labs/
    ├── .env.example
    ├── requirements.txt
    ├── 01_python_basics/
    ├── 02_data_formats/
    ├── 03_rest_api/
    ├── 04_git/
    ├── 05_cisco_apis/
    ├── 06_yang_netconf_restconf/
    ├── 07_docker/
    ├── 08_ansible/
    ├── 09_terraform/
    └── 10_network_troubleshooting/
```

Keep secrets in `labs/.env`, which should never be committed.

---

4. Python virtual environment

Use a venv so Cisco SDKs and lab packages do not collide with other projects.

PowerShell (Windows Python):

```
cd "C:\Users\Reydel\Documents\04_Learning\CCNA Automation"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r labs\requirements.txt
```

If activation is blocked:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Ubuntu (WSL):

```
cd /mnt/c/Users/Reydel/Documents/04_Learning/CCNA\ Automation
python3 -m venv .venv-wsl
source .venv-wsl/bin/activate
python -m pip install --upgrade pip
python -m pip install -r labs/requirements.txt
```

Deactivate with `deactivate`.

Copy environment template:

```
copy labs\.env.example labs\.env
```

Fill in sandbox credentials when you have them. Optional Cisco SDKs (install when you reach Domain 3):

```
python -m pip install meraki dnacentersdk webexpythonsdk
```

---

5. Git setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Create a **private** GitHub repository such as `ccnaauto-labs`. Do not publish API keys.

Local init (only if you want this study folder itself in Git):

```
cd "C:\Users\Reydel\Documents\04_Learning\CCNA Automation"
git init
```

Add a `.gitignore` that excludes:

```
.venv/
.venv-wsl/
labs/.env
__pycache__/
*.pyc
.terraform/
terraform.tfstate
terraform.tfstate.backup
.DS_Store
```

Practice the exam Git operations in `labs/04_git` using a dedicated lab repo, not necessarily this whole study tree.

Git lab (objectives 1.8.a–g and 5.12)

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
echo "lab" > notes.txt
git add notes.txt
git commit -m "Add lab notes"
git switch -c feature/hostname
# edit a file, then:
git add -A
git commit -m "Change hostname URL"
git switch main
git merge feature/hostname
git diff HEAD~1
git push -u origin main
git pull
```

Force a merge conflict: change the same line on `main` and on a branch, merge, resolve, commit. Study `labs/04_git/example.diff` until you can read unified diff headers and `+` / `-` lines.

---

6. Required Python packages

From `labs/requirements.txt`:

Package	Used for
`requests`	REST and RESTCONF (2.9, 3.9, 5.7, 5.10)
`PyYAML`	Parse YAML (1.2), Ansible-like data
`lxml` / stdlib `xml.etree`	XML parsing (1.2, NETCONF)
`xmltodict`	XML → Python dict for NETCONF replies
`ncclient`	NETCONF client (3.8, 5.10)
`python-dotenv`	Load `labs/.env` without hard-coding secrets

Standard library only (no pip): `json`, `unittest`, `ipaddress`, `http.server`.

---

7. Docker setup

From `labs/07_docker`:

```
cd labs\07_docker
docker build -t ccnaauto-lab:1.0 .
docker images
docker run --rm -p 8080:8080 --name ccnaauto ccnaauto-lab:1.0
```

Browse `http://127.0.0.1:8080`. Then:

```
docker ps
docker logs ccnaauto
docker exec -it ccnaauto python --version
docker stop ccnaauto
docker rmi ccnaauto-lab:1.0
```

You must be able to **interpret** `FROM`, `WORKDIR`, `COPY`, `RUN`, `ENV`, `EXPOSE`, `CMD` / `ENTRYPOINT` and explain layers.

---

8. Postman setup

Create a workspace named **CCNAAUTO**.

Create environments:

Environment	Variables
httpbin	`base=https://httpbin.org`
IOS-XE	`host`, `port`, `user`, `pass` from Sandbox
Meraki	`base=https://api.meraki.com/api/v1`, `apiKey`
Catalyst Center	`host`, `user`, `pass`
Webex	`base=https://webexapis.com/v1`, `token`

Starter requests:

1. `GET {{base}}/get?device=edge-01`
2. `POST {{base}}/post` with JSON body `{"hostname":"edge-01"}`
3. `GET {{base}}/basic-auth/admin/s3cret` with Basic Auth
4. RESTCONF:
   `GET https://{{host}}:{{port}}/restconf/data/Cisco-IOS-XE-native:native/hostname`
   Header `Accept: application/yang-data+json`
   Auth: Basic
   SSL verification off

Save the Status code, Headers, and Body panes. The exam will ask you to interpret those three parts (2.6) and troubleshoot from them (2.5).

---

9. Terraform and Ansible smoke tests

Terraform (creates a local JSON file, no cloud bill):

```
cd labs\09_terraform
terraform init
terraform plan
terraform apply -auto-approve
type generated_inventory.json
terraform destroy -auto-approve
```

Ansible (WSL). The sample inventory uses `ansible_connection=local`. Only run the nginx playbook on a Linux VM you own, not blindly against production. To practice **interpretation** without changing a host, read `labs/08_ansible/playbook.yml` and explain each task (package, user, copy, service).

Dry-run syntax check:

```
cd /mnt/c/Users/Reydel/Documents/04_Learning/CCNA\ Automation/labs/08_ansible
ansible-playbook --syntax-check -i inventory.ini playbook.yml
```

---

10. How this lab is used throughout the course

Week / domain	Lab folder	Sandbox or local
Software development, Git, data formats	`01`, `02`, `04`	Local only
APIs, HTTP, Python `requests`	`03`	httpbin / jsonplaceholder
Cisco platforms and SDKs	`05`	DevNet + Webex developer
YANG, NETCONF, RESTCONF	`06`	IOS XE Always-On or reservable
Docker, CI/CD ideas, Bash	`07` + WSL	Local Docker
IaC: Ansible, Terraform, NSO concepts	`08`, `09`	Local + NSO Always-On if available
Network fundamentals / connectivity	`10`	Diagrams + optional CML

DevNet Sandbox workflow

1. Sign in at https://devnetsandbox.cisco.com/
2. Search **IOS XE**, **Catalyst 8000**, **NSO**, **Nexus**, **Meraki**, **Catalyst Center**, or **SD-WAN**.
3. **Always-On** labs are shared. Launch, wait for credentials, copy host/user/password into `labs/.env`. Do not rely on old public passwords from blog posts; many sandboxes now mint **per-session** credentials.
4. **Reservable** labs give a private topology and often require AnyConnect/VPN. Use them when you need write access or a quieter device.
5. Never use Always-On boxes for destructive tests (`write erase`, deleting the management interface, changing AAA so you lock everyone out).

Official sandbox catalog: https://devnetsandbox.cisco.com/

Related free Cisco resources:

Resource	URL	Why
DevNet	https://developer.cisco.com/	API docs, SDKs, Learning Labs
Code Exchange	https://developer.cisco.com/codeexchange/	Sample automation code
Learning Labs	https://developer.cisco.com/learning/	Guided API/NETCONF labs
Network Programmability basics (code)	https://github.com/CiscoDevNet/netprog_basics	RESTCONF/NETCONF examples
IOS XE programmability guide	https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1717/b_1717_programmability_cg/restconf-protocol.html	RESTCONF behavior
NETCONF on IOS XE	https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/26x/26x-programmability-cg/netconf_protocol.html	Port 830, datastores
Meraki API	https://developer.cisco.com/meraki/api-v1/getting-started/	Org/network/device calls
Catalyst Center API	https://developer.cisco.com/docs/catalyst-center/getting-started/	Token + inventory
ACI programmability	https://developer.cisco.com/docs/aci/	APIC object model
Webex APIs	https://developer.webex.com/docs/getting-started	Rooms/messages
pyATS	https://developer.cisco.com/pyats/	Network test framework
CML	https://developer.cisco.com/modeling-labs/	Simulation (5.3)

---

11. Verification checklist

You are ready to study when all of these work:

- [ ] `python --version` prints 3.10 or newer
- [ ] `.venv` is activated and `python -c "import requests, yaml"` succeeds
- [ ] `git --version` works

- [ ] Ubuntu WSL opens and `bash --version` works
- [ ] `docker run --rm hello-world` succeeds
- [ ] Postman can `GET https://httpbin.org/get` and shows status 200
- [ ] You can sign in to DevNet Sandbox
- [ ] `labs/.env` exists and is not committed

Then start **Week 1** in `CCNAAUTO_STUDY_PLAN.md` with `labs/02_data_formats/parse_formats.py`.